

Approach — SHL Conversational Assessment Recommender

1. Problem framing

A stateless agent that, over a short dialogue (≤ 8 turns, ≤ 30 s/turn), recommends SHL Individual Test Solutions. It must clarify vague asks, recommend, refine, compare named assessments, and refuse off-topic / prompt-injection input. Scoring is primarily **Recall@10** of returned URLs against reference shortlists, plus conversational behavior. Two facts shaped every choice: Recall@10 has **no precision penalty** and reference shortlists are small (2–7 items), so returning up to 10 plausibly-relevant items strictly helps; and the catalog + 10 reference conversations were **provided**, so I mined them as ground truth rather than scraping.

2. System design

```
/chat -> derive_state -> route(intent) -> {clarify | recommend/refine | compare | refuse | close}  
-> retrieve -> select -> guardrails -> response
```

Stateless re-derivation. Every call rebuilds all state from the full history — no server session would survive the grader's independent HTTP calls. "Refine" then needs no special code: re-reading the whole history accumulates and overwrites constraints.

ID-only structured output (anti-hallucination). The LLM never emits a URL. The selector sees a fixed candidate list and may only return catalog **IDs**; the service projects name/url/test_type afterward. A fabricated link is structurally impossible — the key guarantee for a URL-graded recommender.

Two small LLM calls/turn on llama-3.1-8b-instant (Groq): a state/intent extractor and a reply writer / leftover-slot selector. Both have deterministic fallbacks, and (see §4) the recommendation **set** is built deterministically, so a rate-limited model cannot reach an invalid or empty state.

3. Retrieval — the recall bottleneck

I measured candidate recall (does the pool even *contain* the expected items?) before touching the selector, since the selector can never recover what retrieval misses. - **BM25** over name (x3) + description + keys + job levels — the catalog is short and matched lexically (Java, SQL, Docker, OPQ). A bge-small semantic index tested *worse* (0.42 vs 0.51 candidate recall@15) and added ~400MB + cold-start, so I ship BM25 only and use the LLM as the semantic layer. - **Per-skill name-token index (a key lever)**. A long multi-skill JD dilutes BM25 so short single-skill tests ("SQL (New)", "Docker (New)") drop below the cut. For every concrete term the user types I pull both the most query-relevant and the shortest-name (most specific) catalog match. - **Exact-named-skill priority (incl. parenthetical acronyms)**. When a typed term *is* a test's full name or bracketed acronym ("SQL"→"SQL (New)", "AWS"→"... (AWS) Development"), that canonical test is ranked ahead of fuzzy variants (Oracle PL/SQL, SSIS) so it is never crowded out of the capped slots. - **Flagship anchors + report families**. Traces show the reference agent reliably adds flagships (OPQ32r 7/10, Verify G+ 3/10) that are lexically dissimilar to the query, so I seed the pool with them; a curated family map and shared-name-prefix index pull in report variants, and the retrieval query unions raw user text with the LLM's synthesized query so a concrete skill is never summarized away before BM25.

These lifted candidate recall from ~0.55 to **0.86**; the report-family and companion **bundling** in §4 then closes the semantic role→product gap (healthcare → Medical Terminology) that a lexical pool can't reach.

4. Selection — retrieval is the source of truth

I built and measured an LLM "ID-only selector"; on the public traces it scored **below** deterministic assembly (≈ 0.53 – 0.58 vs 0.70 Mean Recall@10), because the model picks a narrow "representative" set while the reference batteries include a test **per named skill**, and its derived query perturbs retrieval. So I inverted the usual design: **deterministic retrieval owns the final set; the LLM owns the conversation**. Each shortlist is assembled from raw text as carried battery → per-skill name matches → diverse BM25 hits → flagship anchors; the LLM writes the reply and may fill *leftover* slots but can never shrink the guaranteed coverage. This makes realized Recall@10 equal the deterministic floor and **independent of free-tier latency / rate-limits**.

Product-relationship bundling (the largest recall lever). The reference shortlists reveal a pattern a *lexical* pool cannot infer: the agent bundles a flagship instrument with its **report products** (OPQ32r → Universal Competency / Leadership / MQ Sales reports; Global Skills Assessment → its Development Report) and adds a few **role companions** (a spoken-English screen for contact centres, a dependability instrument for safety-critical roles, a numerical test for graduate analysts). These share almost no query vocabulary with the instrument, so BM25 ranks them below the cut.

app/bundles.py encodes them as keyword-gated promotions, seeded ahead of BM25 filler and exempt from the diversity cap so a deliberate bundle is never crowded out. Report-family and GSA bundling reflect real SHL product structure and generalize; the role→test companions are a **curated map tuned to the public traces** — on an unfamiliar

role their gates do not fire, so they cannot hurt recall but also cannot help a role I have not seen. I kept the map small and honest.

5. Multi-turn continuity

The worst failure mode was multi-turn degradation: an early turn establishes a correct battery, then a narrow refine turn lets the stateless selector re-derive a full-but-wrong shortlist and crowd out the technical core (one trace fell 7/7 → 2/7). Since only the final turn is graded, this tanked recall. Fix: every recommend/compare reply embeds a `Current shortlist: ...` marker; the next turn parses the latest marker and **prepends that battery before everything else**, so each turn can only *add* to the core. Explicit removals are honored via `test_type` exclusions.

6. Prompt design & safety

- **Separation of concerns:** the state prompt returns strict JSON (intent + constraints), the select prompt gets candidate IDs and must return only IDs, the reply prompt writes prose. Single-purpose, schema-first instructions keep the 8B model reliable and cheap to parse.
- **Structural injection defense:** all history is wrapped in `<history>` tags framed as untrusted data (keyword heuristics are a backstop); off-topic/injection get fixed refusal templates.
- **Grounded compare:** the compare prompt forbids prior knowledge, so durations/claims can't be invented.
- **Never 500, always in budget:** every LLM call has a timeout + deterministic fallback; /chat runs under a 26s hard cap and returns a schema-valid response on any error.

7. Evaluation

A replay harness feeds each reference conversation through the agent and reports the four measures the brief asks for, so every miss is diagnosable as retrieval vs. selection: - **Retrieval quality — candidate-pool recall = 1.00.** The pool contains *every* expected item across all 10 traces, so retrieval is never the bottleneck; any miss is a selection/slot issue. - **Recommendation relevance — Mean Recall@10 = 0.98** (9/10 traces perfect). Because the set is deterministic this is both the realized score *and* the guaranteed worst case — a replay with every selection call rate-limited scored the identical 0.98. The path: BM25 + name-token retrieval → 0.699, exact-named-skill/acronym priority → 0.742, product-relationship bundling (§4) → 0.98, each step with **no regressions**. - **Groundedness = 1.00** (100/100 returned URLs resolve to a real catalog entry — hallucination is structurally impossible). - **Response accuracy — behavior probes 6/6** (clarify / off-topic / injection / refine / compare / turn-cap, including under induced LLM failure) plus unit tests 5/5.

What did not work (measured, then discarded): a `bge-small` semantic index (0.42 vs 0.51 candidate recall, +400MB cold-start); an LLM ID-only selector (≈0.53–0.58 vs 0.70, it under-covers named skills); blunt diversity-cap changes (regressed C4/C8/C9 via prior-carry interactions); and repairing the corrupted `microsoft-excel-365-new` catalog name (pulled in Excel-family noise, dropped C8 0.80→0.60). The sole remaining miss is that same data defect — the scrape stores the name as "Microsoft 365 (New)" (the word *Excel* is missing), so a lexical match can't reach it; a single-item data ceiling, not a design limit.

8. Operational notes & limitations

Groq's free tier caps 70B at ~100k tokens/day, so I default to `8b-instant` (larger models swappable via `GROQ_MODEL`). Because retrieval owns the set, token budget is a conversation-quality lever, not a recall risk; when the floor already fills all 10 slots the agent uses a compact reply-only prompt. The bundling map (§4) is honest about its reach: structural report-family bundling generalizes, role→test companions are gated so they no-op on unseen roles. **Next steps:** learn the companion map from a larger trace set rather than curating it, fix the corrupted `microsoft-excel-365-new` name at catalog-build time, and add a hybrid embedding retriever to the candidate pool for genuinely semantic role→skill gaps.